أكاديمية سدايا
SDAIA Academy

SDAIA
الهيئة السعودية للبيانات والذكاء الاصطناعي
Saudi Data & AI Authority

SDAIA ACADEMY · APPLIED GENERATIVE AI

Production and the numbers

Day 4 · Wednesday 5 August 2026 · Musa Ibn Rashid

What does this cost per user per month?

PRESENTER NOTES

Today we stop adding capability and start measuring what you already have.

By noon you will have a table of your own numbers: latency per request, tokens in and out, cost, and a measured cache hit rate.

Engineers who can answer the question on this slide get hired. That is not a joke — it is the single most useful thing you will take away this week.

أكاديمية سدايا
SDAIA Academy

SDAIA

THE WEEK

Where we stand

SUNDAY	~~From token to typed output — the model as a reliable function.~~
MONDAY	~~Retrieval you can measure — RAG, hybrid search, and a golden set that scores it.~~
TUESDAY	~~Tools and agents — the model requests, your code executes.~~
WEDNESDAY	Production and the numbers — latency, caching, and what this costs per user.
THURSDAY	Break it, then defend it — prompt injection, red teaming, and your demo.

> **There is no such thing as a dumb question.** If you are wondering, someone else is too — ask.

PRESENTER NOTES

Three days of building. Today we stop adding capability and start measuring what you already have.

Nothing new gets built today that you did not already have — it gets instrumented instead.

Tomorrow morning you attack each other's systems, so whatever runs today is what gets tested.

أكاديمية سدايا SDAIA Academy

SDAIA

TODAY

How the day runs

9:15 – 10:05	**Block 1 · The gap and the arithmetic · Cost Auction**	50 min
10:05 – 10:20	Break	
10:20 – 11:00	Block 2 · Reliability and observability	40 min
11:00 – 11:20	Break	
11:20 – 12:00	Block 3 · Notebook 4 — instrument your own project	40 min
12:00 – 1:00	Lunch	60 min
1:00 – 2:00	Block 4 · Project build	60 min
2:00 – 2:00	Break	
2:00 – 2:30	Block 5 · README clinic, then Thursday briefing	30 min

PRESENTER NOTES

Block 3 runs against your own project, so it needs to be working before lunch.

The README clinic at two o'clock is not optional — fifteen rubric points sit in the repository.

We finish with the Thursday briefing so nobody is guessing about the format overnight.

DAY 4

What you will be able to do by 12:00

01 Name the concrete differences between your notebook and a production system.

02 Compute what an assistant costs per user per month, from token counts.

03 Add retries with backoff, a cache, and a fallback chain to your own code.

04 Measure and report latency, tokens, cost and cache hit rate from real runs.

05 Say what you would log, trace and alert on before you let anyone use it.

PRESENTER NOTES

Number two is the one your management will ask you first, and almost nobody in this field can answer it without going away for a week.

Number four is what you will paste into your Thursday presentation. A real table of your own numbers is far more persuasive than any claim about quality.

أكاديمية سدايا
SDAIA Academy

SDAIA

YESTERDAY · QUESTIONS TO THE ROOM

Three questions before we start

01 Who executes a tool call?

02 Why does step twenty of an agent loop cost more than step one?

03 What does a tool return when it fails, and why?

> Five minutes. Then we start counting things.

PRESENTER NOTES

The second question is the bridge into today, because it is the first cost-shaped question of the week.

If nobody gets it: the conversation history grows with every step, and you re-send all of it every time.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART ONE

The gap

Everything that is fine in a notebook and not fine with five hundred users.

PRESENTER NOTES

Your projects currently work. That is genuinely an achievement and it is also the easiest part.

This next section is the list of things that break the first week something real is in front of real people.

أكاديمية سدايا SDAIA Academy

SDAIA

THE GAP

Prototype vs production

	YOUR NOTEBOOK	PRODUCTION
Users	One. You.	Hundreds, at the same moment, at 9am
Errors	You re-run the cell	Handled, retried, logged, or shown honestly
Cost	Ignored — it's free	Measured, budgeted, alerted on
Latency	You wait. It's fine.	Eight seconds of silence loses the user
Secrets	In a Colab secret	In a secret manager, rotated, never in git
Change	Edit and re-run	Versioned, reviewed, evaluated before release

> None of this is AI-specific. It is ordinary engineering that AI projects skip because the demo was so easy.

PRESENTER NOTES

Read the last line slowly. The reason generative AI projects fail in production is almost never the model.

It is that a two-day prototype convinced everyone the hard part was done, so nothing on the right-hand column was ever budgeted.

Ask the room which row they think will bite them first. It is usually cost or latency.

أكاديمية سدايا
SDAIA Academy

SDAIA

SCALE

Four things that decide whether it holds up

Concurrency
Fifty people at 9am is fifty simultaneous calls. One synchronous loop serves them one at a time.

Async
These calls are almost entirely waiting. Async lets one process hold hundreds of open requests.

Caching
The same question, asked by forty people. Answer it once. Free and instant thereafter.

Rate limiting
Yours from the provider, and the one you impose on your own users so one script cannot drain the budget.

> Caching is the cheapest of the four and the one most often left out.

PRESENTER NOTES

The async point is worth a sentence of explanation: your process is not computing anything, it is sitting on a socket waiting for a model, so blocking is pure waste.

Rate limiting in both directions is the part people forget. You need a limit on your users, not only a plan for the provider's limit on you.

أكاديمية سدايا
SDAIA Academy

SDAIA

CALLBACK · MONDAY

Those 429s were the lesson

On Monday, when everyone embedded their chunks at once, the room filled with `429 RESOURCE_EXHAUSTED`.

That was a rate limit, and you already know how it feels. Your users will feel the same thing unless you design for it.

What to do about it

- **Batch** — fewer, larger requests.
- **Backoff** — wait 1s, 2s, 4s. Never retry immediately in a loop.
- **Queue** — smooth the burst instead of amplifying it.
- **Degrade honestly** — "busy, try again in a moment" beats a spinner that never resolves.

> A retry without backoff is not a fix. It is a denial of service attack on yourself.

PRESENTER NOTES

This is why we hit the rate limit deliberately on Monday rather than quietly avoiding it — lived experience beats a definition on a slide.

The last line is the one to say firmly. Tight retry loops turn a brief limit into a sustained outage, and I have watched it happen.

أكاديمية سدايا
SDAIA Academy

SDAIA

PART TWO

The arithmetic

Tokens in, plus tokens out, times a price. That is the whole model.

PRESENTER NOTES

This next twenty minutes is the part of the course that most directly changes how you are seen at work.

Nobody teaches this, and it is arithmetic. That combination is unusual and worth exploiting.

COST

What you are actually paying for

Every request has two meters:

- **Input tokens** — your system prompt, the retrieved chunks, the conversation so far, the question.
- **Output tokens** — what it writes back. Usually priced higher.

In a RAG system the *retrieved chunks dominate the input*. Four chunks of 500 tokens is 2,000 tokens on every single call, before anyone types a word.

Where the money actually goes

- Top-k too high — you pay for every chunk, every time.
- Long system prompts — re-sent on every call, forever.
- Agent loops — history re-sent at every step.
- No cache — the same question paid for forty times.

PRESENTER NOTES

Point at the retrieved-chunks line. This is the direct consequence of Monday's top-k decision, and it is why "just raise k" is an expensive habit.

The system prompt point surprises people: a four-hundred-token system prompt is small until you multiply it by forty thousand requests a month.

COST · LIVE

Work it out on the screen

Users per day	500
Queries per user	4
Avg input tokens	2200
Avg output tokens	400
$ per 1M input	0.10
$ per 1M output	0.40
Working days / month	22
Cache hit rate %	0

Prices are per million tokens. Put today's real figures in
— these are placeholders.

PER QUERY	$0.00038
PER DAY	$0.76
PER MONTH	$16.72
PER USER / MONTH	$0.03

PRESENTER NOTES

Type real numbers into this while they watch. Start with the defaults, then change one thing at a time so they see what moves the total.

Drop the input tokens from twenty-two hundred to twelve hundred — that is retrieving two chunks instead of four — and watch the monthly figure fall by nearly half.

Then put the cache hit rate to forty percent and watch it fall again. Those two numbers are the whole cost conversation.

CALLBACK · SUNDAY

The same product costs more in Arabic

On Sunday we measured it: the same sentence is two to three times the tokens in Arabic.

Both meters are affected. Arabic documents make bigger chunks, so the input grows. Arabic answers are more tokens, so the output grows.

Put a bilingual assistant into the calculator and the monthly figure moves materially.

What to do about it

- Budget for it explicitly rather than discovering it in month two.
- Cache harder — repeated Arabic questions save more per hit.
- Keep chunks tighter for Arabic sources.
- Measure both languages separately in your logs.

PRESENTER NOTES

Nobody teaches this and it is directly relevant to every project in this room.

If you present a cost estimate to management based on English measurements for a service that will run in Arabic, you will be wrong by a factor that gets noticed.

The last bullet is practical: log the language, so you can see the split rather than guessing at it.

أكاديمية سدايا
SDAIA Academy

SDAIA

LATENCY

Four ways to make it faster, and one to make it feel faster

Right-size the model
Most requests do not need your best model. Route the easy ones to the cheap fast one.

Cache
A hit is zero latency and zero cost. Nothing else on this slide competes with that.

Shorter prompts
Fewer input tokens is less time to first token. Top-k discipline pays twice.

Batch offline work
Embedding, summarising, indexing — none of it belongs on the user's request path.

> **Streaming** changes none of the above. It changes when the first word appears — and users read that as speed.

PRESENTER NOTES

Separate the four real fixes from streaming very clearly, because people conflate them constantly.

Streaming does not reduce total generation time by a millisecond. It moves time-to-first-token from about six seconds to about three hundred milliseconds, and that is what the user experiences as fast.

In the notebook you run both side by side with a timer, and the total is the same. Watch for the surprise.

CODE

Streaming, and a cache that pays for itself

```
# Streaming: same total time, first word in ~300ms.
for chunk in client.models.generate_content_stream(
        model=MODEL, contents=prompt):
    print(chunk.text, end="", flush=True)

# A cache. Twelve lines, and the best value in the file.
import hashlib, json

CACHE = {}

def cached_ask(prompt, **kw):
    key = hashlib.sha256(
        (prompt + json.dumps(kw, sort_keys=True)).encode()
    ).hexdigest()

    if key in CACHE:
        stats["hits"] += 1
        return CACHE[key]

    stats["misses"] += 1
    CACHE[key] = ask(prompt, **kw)
    return CACHE[key]
```

Three notes

- Key on the prompt *and* the settings — a different temperature is a different question.
- Count hits and misses from the start. An unmeasured cache is a guess.
- In production this is Redis with a time-to-live, not a dict — but the shape is identical.

A cache with no expiry serves yesterday's policy after it changed. Set a TTL.

أكاديمية سدايا
SDAIA Academy

SDAIA

RELIABILITY

Assume every call can fail

The four controls

- **Timeout** — never wait forever. Decide the number.
- **Retry with backoff** — 1s, 2s, 4s, then give up.
- **Fallback chain** — primary model, then a cheaper one, then a canned response.
- **Circuit breaker** — if it is failing for everyone, stop trying for a minute.

Your users will forgive a degraded answer. They will not forgive a page that hangs.

Retry only what is worth retrying

A 429 or a 503 will probably succeed in two seconds. A 400 will fail identically forever, and retrying it wastes your budget and your user's time.

The canned response matters. "I can't reach the assistant right now — here is the policy page" is a better product than a spinner.

PRESENTER NOTES

The distinction between retryable and non-retryable errors is the difference between a robust system and an expensive one.
The fallback chain is easy to build and almost never built. Two lines of code turn a total outage into a slightly worse answer.
In the notebook you break the primary model on purpose and watch the fallback engage.

CODE

Retry, then fall back

```
import time, functools

RETRYABLE = (429, 500, 502, 503, 504)

def retry(tries=3, base=1.0):
    def deco(fn):
        @functools.wraps(fn)
        def wrapper(*a, **kw):
            for n in range(tries):
                try:
                    return fn(*a, **kw)
                except ApiError as e:
                    # Only retry what might succeed next time.
                    if e.code not in RETRYABLE or n == tries - 1:
                        raise
                    time.sleep(base * 2 ** n)  # 1s, 2s, 4s
        return wrapper
    return deco

def ask_with_fallback(prompt):
    for model in (PRIMARY, CHEAPER):
        try:
            return ask(prompt, model=model)
        except Exception:
            log.warning("falling back from %s", model)
    return CANNED   # honest, useful, never a spinner
```

Two decisions in this file

- **Which errors to retry.** A 400 will fail identically forever — retrying it wastes budget and time.
- **What the last resort is.** A canned response with a link beats an error page.

In the notebook you break the primary on purpose and watch the fallback engage.

أكاديمية سدايا
SDAIA Academy

SDAIA

WHAT "MEASURED" LOOKS LIKE

The table you will produce today

METRIC	YOUR RUN	WHY IT MATTERS
Median latency	2.1 s	What a typical user waits. Optimise the median first.
p95 latency	6.8 s	The bad experience. This is what people complain about.
Avg input tokens	2,240	Mostly retrieved chunks. Directly set by your top-k.
Avg output tokens	380	Priced higher than input. Shorter answers are cheaper answers.
Cache hit rate	38%	Measured, not hoped for. Every hit is free and instant.
Cost per query	$0.00037	Multiply by traffic and you have the monthly number.

> Six numbers. With these you can answer any question your management asks about this system.

PRESENTER NOTES

These are illustrative figures — yours will differ, and the point is that you will have your own by noon.

Explain the median-versus-p95 distinction, because it is the single most useful idea in performance work and almost nobody arrives knowing it.

Put this table in your presentation tomorrow. Pairs who show measured numbers read as engineers; pairs who say "it feels fast" do not.

OBSERVABILITY

If you cannot see it, you cannot run it

Logging
Every request: prompt id, tokens in and out, latency, model, cache hit, cost, outcome.

Tracing
One request across every step — retrieval, each tool call, generation — as a single timeline.

Metrics
p50 and p95 latency, error rate, cost per day, cache hit rate, tokens per request.

Continuous eval
Run your golden set on a schedule. Documents drift, models change, quality moves.

> Tools that do this for you: **LangSmith**, **Langfuse**, **Phoenix**. Or a dataframe and a plot, which is what you build today.

PRESENTER NOTES

Tracing is the one that is specific to this kind of system. A single user question can become five model calls, and without a trace you cannot tell which one was slow or wrong.

Continuous evaluation is Monday's golden set, run on a schedule. That is the whole idea, and it is why we spent the afternoon building it.

Name the tools but do not sell them. Today you build the small version yourself, which is enough to understand what those products are for.

أكاديمية سدايا
SDAIA Academy

SDAIA

ARCHITECTURE

What the whole thing looks like deployed

USER INTERFACE
Web or chat. Streaming, citations, honest failure states.

API LAYER
Auth, rate limiting per user, request logging.

ORCHESTRATION
Prompts, the agent loop, retries, guardrails.

CACHE
Checked before any model call is made.

MODEL
Primary and fallback. Swappable behind one interface.

VECTOR STORE
Your chunks and metadata.

TOOLS
Internal APIs, least privilege.

MONITORING
Logs, traces, metrics, alerts.

GOVERNANCE
Audit trail, PII controls, evaluation runs.

> The model is one box. Notice how much of this diagram is ordinary systems engineering.

PRESENTER NOTES

Walk the request through the diagram once, out loud, from the top: interface, API, cache check, orchestration, retrieval, model, back up with a citation.

Then make the point on the callout. One box on this whole diagram is the model, and it is the box you will spend the least time on.

For your projects, none of this is required. It is the map for what happens after this week.

أكاديمية سدايا
SDAIA Academy

SDAIA

THE PRODUCT

UX for something that is sometimes wrong

Do

- **Stream** — so it feels alive within a second.
- **Cite** — source and page, clickable. Trust comes from checkability.
- **Say "I don't know"** — and mean it, when retrieval came back empty.
- **Show the failure** — plainly, with something useful to do next.

Don't

- Hide that an AI produced it.
- Present an unsourced answer with the same confidence as a sourced one.
- Leave a spinner running when the call has already failed.
- Make the user retype the question after an error.

Citations are not decoration. They are how a user decides whether to believe you.

PRESENTER NOTES

In a government context the citation is the feature. An answer someone can verify in ten seconds is worth more than a better-written answer they cannot check.

The "I don't know" behaviour needs building deliberately — it does not happen by default, and it is the thing that makes the rest of the answers credible.

LAB · NOTEBOOK 4

Instrument your own project

day4_production.ipynb

- A retry decorator with backoff, and a forced failure to prove it fires.
- Streaming against non-streaming, timed.
- A cache, then 20 queries with 8 repeats — measure the hit rate.
- Log every request into a DataFrame and `describe()` it.
- Plot latency and cumulative cost. Fill in the cost formula yourself.

This one runs against *your* project

Not a toy. Import your own retrieval and agent functions from the last two days.

You finish with a table of your own numbers.
That table goes in tomorrow's presentation.

Open the notebooks →

PRESENTER NOTES

The cost formula is deliberately left blank. Fill it in from the real pricing page, because reading a pricing page is a skill and it changes every few months.

If your project is not running yet, come and find me now rather than at one o'clock, because this notebook needs something to instrument.

أكاديمية سدايا
SDAIA Academy

SDAIA

THIS AFTERNOON

Build, then the README clinic

1:00 – 2:00 · Build

I circulate. Get the numbers table out of Notebook 4 and into your repository.

2:00 – 2:20 · README clinic

Every pair writes their README **now**, with me checking. All six SDAIA requirements on screen.

READMEs written the night before are always bad. This is not negotiable.

2:20 · Tomorrow's format: four minutes, two for questions. An honestly-explained broken demo scores better than a fake working one.

PRESENTER NOTES

Be firm about the README clinic. Fifteen points of the rubric are repository quality, and it is the cheapest fifteen points available.

Say the last line clearly and then say it again tomorrow morning. Someone's demo will break, and I want them to know in advance that explaining it honestly is worth more than pretending.

أكاديمية سدايا
SDAIA Academy

SDAIA
الهيئة السعودية للبيانات والذكاء الاصطناعي
Saudi Data & AI Authority

DAY 4

What today was

1 · The gap between a notebook and a system is ordinary engineering — and it is where these projects die.

2 · Cost is tokens in, plus tokens out, times a price. You can now answer it for your own project.

3 · Retries, caching and fallbacks are a day's work and they change what you can promise.

4 · **You measured your own system.** Not estimated — measured.

PRESENTER NOTES

Four things, and the fourth is what separates you from most people who will claim this on a CV.

Tomorrow morning we try to break each other's systems, so bring yours running and bring your sense of humour.

Post-test is at quarter past nine, first thing. Do not be late for it.